

MeetRooms — Documentation Technique de Déploiement

Application web interne de réservation de salles de réunion pour le groupe AssurNova.

Prérequis

Outil	Version minimale
Docker	24+
Docker Compose	v2 (plugin intégré)
Node.js / npm	18+ (compilation SCSS)
Composer	2+
PHP	8.2+

Architecture

```
├─ Dockerfile                # Image PHP 8.2 + Apache
├─ docker-compose.dev.yml    # Environnement dev      (port 8084)
├─ docker-compose.recette.yml # Environnement recette (port 8085)
├─ docker-compose.prod.yml   # Environnement prod   (port 8086)
├─ .env.example              # Template de variables d'environnement
├─ sql/init_db.sql           # Schéma et données initiales
├─ scripts/
│   └─ setup.sh              # Installation des dépendances
│   └─ start.sh              # Lancement interactif d'un environnement
│   └─ deploy.sh             # Déploiement prod (pull + rebuild)
```

Image Docker

L'image est basée sur `php:8.2-apache`. Elle :

- Active `mod_rewrite`
- Définit `/var/www/html/public` comme `DocumentRoot`
- Installe les extensions PHP `pdo`, `pdo_mysql`, `zip`
- Embarque le code source dans l'image (pour recette et prod)

En **dev**, le code source est monté en volume (- `./var/www/html`), ce qui permet l'édition à chaud sans reconstruire l'image.

Installation initiale

```
# Cloner le dépôt
git clone <url-du-repo> && cd meetrooms
```

```
# Installer les dépendances PHP et frontend
./scripts/setup.sh
```

Ce script exécute `npm install` (Sass) et `composer install`.

Variables d'environnement

Copier `.env.example` pour chaque environnement et adapter les valeurs, notamment les mots de passe en production :

```
cp .env.example .env.dev
cp .env.example .env.recette
cp .env.example .env.prod    # Remplacer les CHANGE_ME
```

Variables disponibles :

Variable	Description
PORT	Port exposé par le conteneur web
DB_HOST	Nom du service base de données (ex: db_dev)
DB_NAME	Nom de la base de données
DB_USER	Utilisateur de la base de données
DB_PASSWORD	Mot de passe de l'utilisateur
MYSQL_ROOT_PASSWORD	Mot de passe root MariaDB
MYSQL_DATABASE	Nom de la base à créer au démarrage
MYSQL_USER	Utilisateur à créer au démarrage
MYSQL_PASSWORD	Mot de passe de cet utilisateur

`DB_HOST` doit correspondre au nom du service dans le `docker-compose` (ex: `db_dev`, `db_recette`, `db_prod`).

Lancement des environnements

Méthode rapide (script interactif)

```
./scripts/start.sh
# Choisir : dev | recette | prod | tous
```

Méthode manuelle

```
# Développement
docker compose --file docker-compose.dev.yml --env-file .env.dev up -d --build

# Recette
docker compose --file docker-compose.recette.yml --env-file .env.recette up -d --build

# Production
docker compose --file docker-compose.prod.yml --env-file .env.prod up -d --build
```

Les trois environnements peuvent tourner **simultanément** sur la même machine :

Environnement	URL locale	Particularité
dev	http://localhost:8084	Code monté en volume (hot reload)
recette	http://localhost:8085	Code embarqué dans l'image
prod	http://localhost:8086	Code embarqué dans l'image

Arrêter un environnement

```
docker compose --file docker-compose.<env>.yml down
# Ajouter -v pour supprimer également les volumes (données)
```

Persistence des données

Chaque environnement dispose d'un volume Docker nommé dédié :

Volume	Environnement
db_dev_data	dev
db_recette_data	recette
db_prod_data	prod

Le schéma initial est chargé automatiquement depuis `sql/init_db.sql` au premier démarrage du conteneur base de données.

GitFlow

feature/* → dev → recette → prod

Branche	Rôle	Push direct
dev	Intégration quotidienne	Oui
recette	Validation pré-production	Interdit — PR depuis dev

prod	Production stable	Interdit — PR depuis recette
------	-------------------	------------------------------

Règles de protection (prod) :

- Pull Request obligatoire avec au moins 1 approbation
- Le workflow CI Run Tests & Build doit être vert
- Force push interdit, y compris pour les administrateurs

Règles de protection (recette) :

- Pull Request obligatoire
- Le workflow CI Run Tests & Build doit être vert (mode strict : `--fail-on-warning`)

Pipeline CI

ci.yml — Tests automatiques

Déclenché sur push et PR vers dev , recette , prod .

Étapes :

1. Démarrage d'un service MariaDB 10.5
2. Installation des dépendances PHP (`composer update`)
3. Compilation SCSS (`npm install && npm run build`)
4. Initialisation de la base de test (`sql/init_db.sql`)
5. Exécution des tests PHPUnit
 - Sur recette et prod : mode strict (`--fail-on-warning --fail-on-risky`)
 - Sur dev : mode standard

recette.yml — Build et push de l'image Docker

Déclenché sur push vers recette .

Étapes :

1. Connexion au DockerHub (secrets DOCKER_HUB_USER / DOCKER_HUB_TOKEN)
2. Build de l'image depuis le Dockerfile
3. Push avec deux tags :
 - `<user>/meetrooms-web:latest`
 - `<user>/meetrooms-web:<sha-du-commit>`

Secrets GitHub requis :

Secret	Description
DOCKER_HUB_USER	Nom d'utilisateur DockerHub
DOCKER_HUB_TOKEN	Token d'accès DockerHub
DB_NAME	Nom de la base (CI)
DB_USER	Utilisateur de la base (CI)
DB_PASSWORD	Mot de passe (CI)

DB_ROOT_PASSWORD	Mot de passe root (CI)
------------------	------------------------

Déploiement en production

```
./scripts/deploy.sh
```

Ce script effectue un `git pull origin prod` puis relance les conteneurs via `docker-compose.prod.yml`.

L'application est accessible sur <http://localhost:8086>.

Tests

```
# Lancer tous les tests
./vendor/bin/phpunit tests/

# Mode strict (équivalent recette/prod)
./vendor/bin/phpunit --fail-on-warning --fail-on-risky tests/
```

Les tests unitaires couvrent notamment la détection de conflits de réservation (chevauchement d'horaires).